LAB TICKETRUSH: BÁN VÉ CONCERT

OUTPUT:

```
=== KIỂM TRA BẢN NAIVE (GET rồi SET) ===
winners (mong đợi 1, nhưng thực tế sẽ thấy nhiều hơn): 12

=== KIỂM TRA BẢN NX (Nguyên tử) ===
winners (luôn luôn là 1): 1

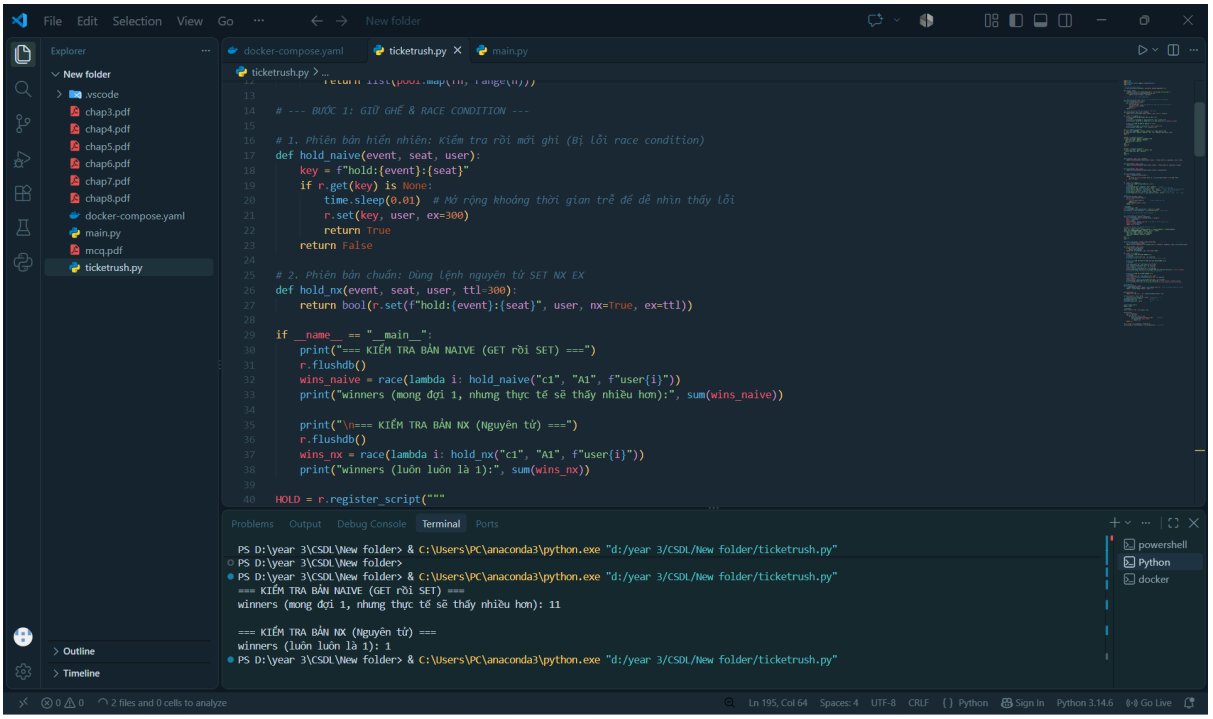
=== KIỂM TRA KỊCH BẢN BƯỚC 2 ===
linh giữ A1: True
nam giữ A1: False
nam xác nhận A1: False
linh xác nhận A1: True
nam giữ lại A1: False
Trạng thái sơ đồ ghế: {'A1': 'sold', 'A2': 'free'}
100 0 100

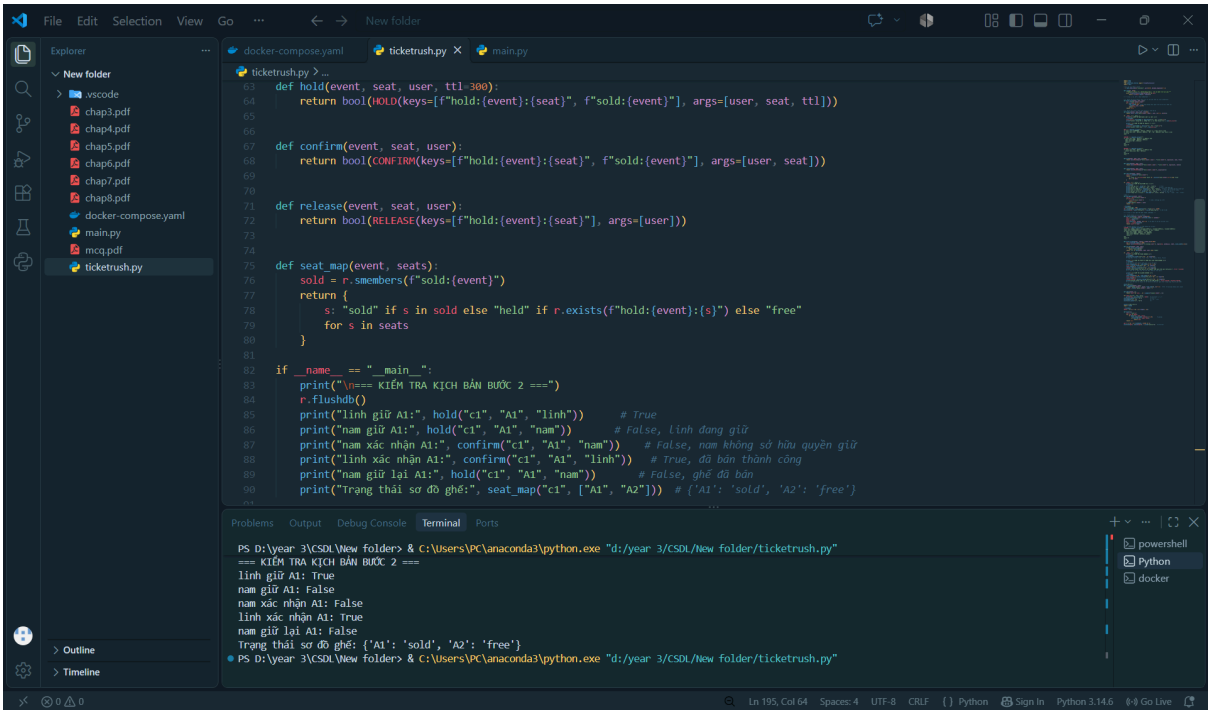
=== KIỂM TRA FIXED WINDOW ===
[True, True, True, True, True, False, False, False]

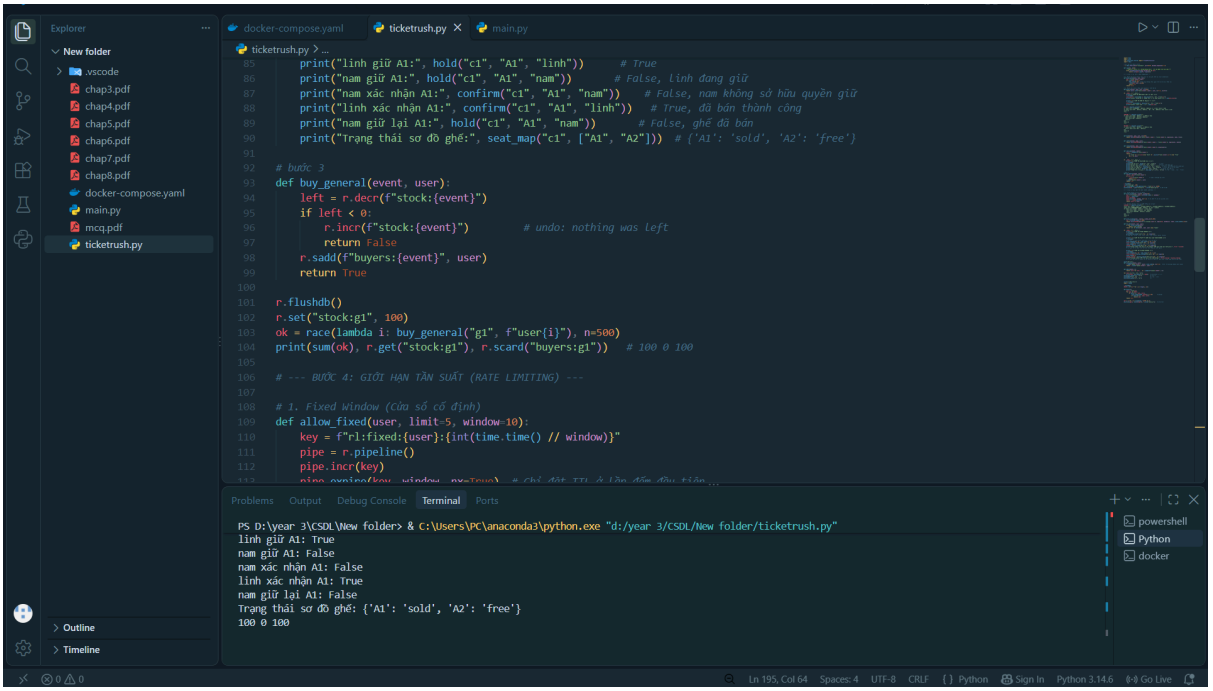
=== KIỂM TRA ĐIỂM YẾU RANH GIỚI CỦA FIXED WINDOW ===
Fixed window (Cho phép cả 10 request vượt giới hạn qua ranh giới): [True, True, True, True, True,
True, True, True, True, True]

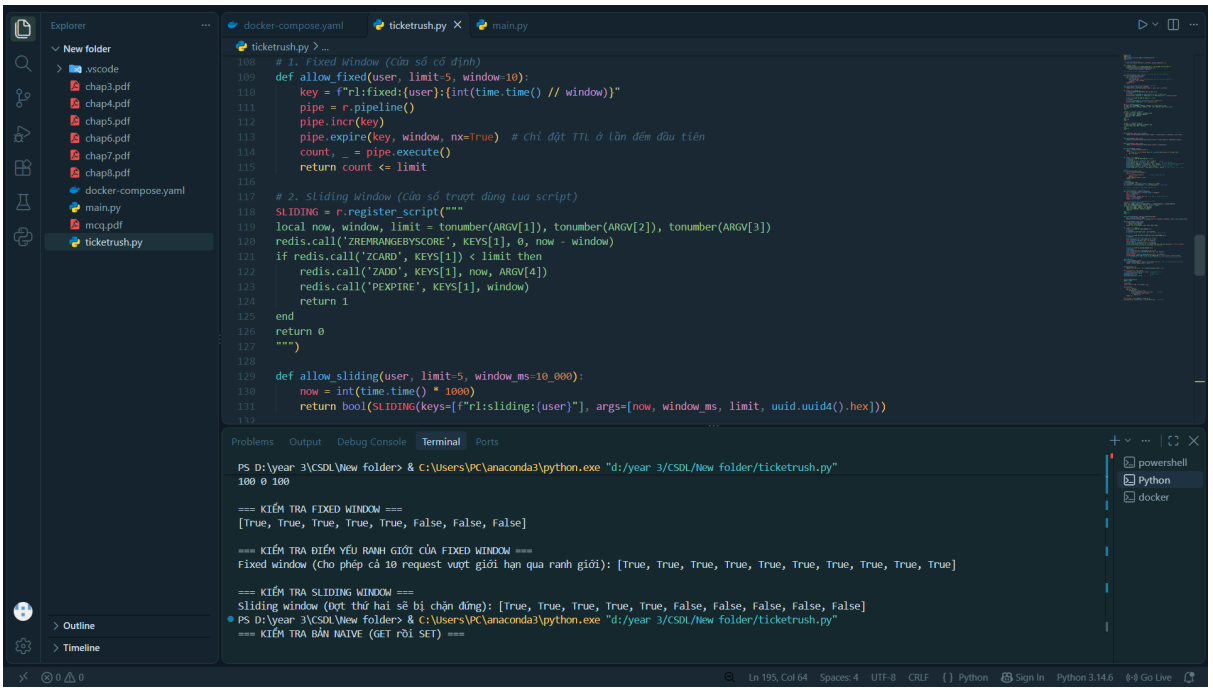
=== KIỂM TRA SLIDING WINDOW ===
Sliding window (Đợt thứ hai sẽ bị chặn đứng): [True, True, True, True, True, False, False, False,
False, False]
BƯỚC 5
u1 1
u2 2
u3 3
1
['u1', 'u2']
1
===BƯỚC 6===
20 20 20
```

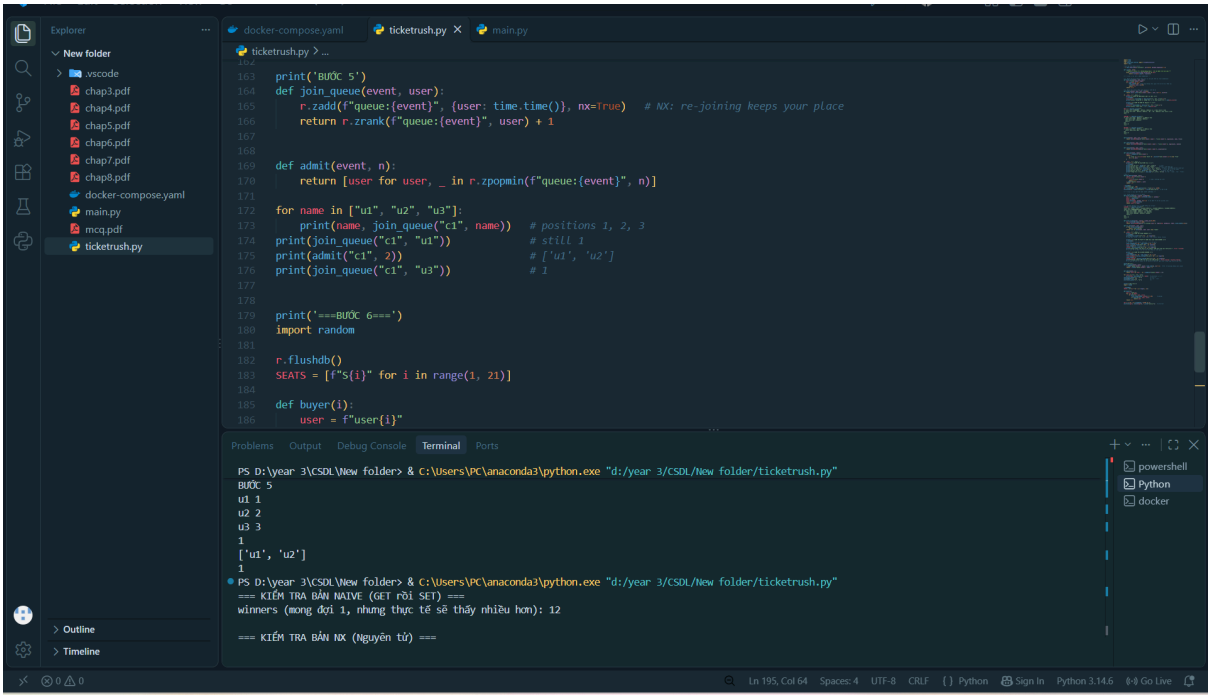
Bước 1:

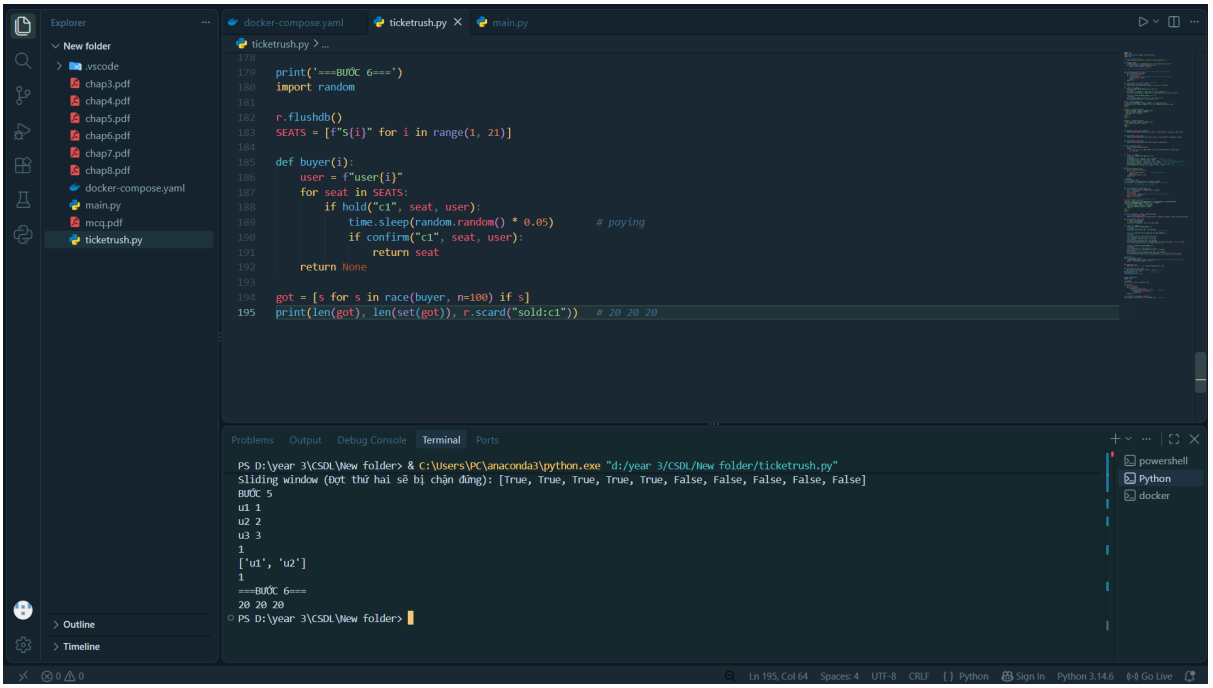

File Edit Selection View Go
New folder
Explorer
docker-compose.yaml
ticketrush.py
main.py
New folder
.vscode
chap3.pdf
chap4.pdf
chap5.pdf
chap6.pdf
chap7.pdf
chap8.pdf
docker-compose.yaml
main.py
mcq.pdf
ticketrush.py
Outline
Timeline
ticketrush.py > ...
--- BƯỚC 1: GIỮ GHẾ & RACE CONDITION ---
1. Phiên bản hiện nhiện: Kiểm tra rồi mới ghi (Bị Lỗi race condition)
def hold_naive(event, seat, user):
key = f"hold:{event}:{seat}"
if r.get(key) is None:
time.sleep(0.01) # Mở rộng khoảng thời gian trễ để dễ nhìn thấy lỗi
r.set(key, user, ex=300)
return True
return False
2. Phiên bản chuẩn: Dùng Lệnh nguyên tử SET NX EX
def hold_nx(event, seat, user, ttl=300):
return bool(r.set(f"hold:{event}:{seat}", user, nx=True, ex=ttl))
if __name__ == "__main__":
print("=== KIỂM TRA BẢN NAIVE (GET rồi SET) ===")
r.flushdb()
wins_naive = race(lambda i: hold_naive("c1", "A1", f"user{i}"))
print("winners (mong đợi 1, nhưng thực tế sẽ thấy nhiều hơn):", sum(wins_naive))
print("\n=== KIỂM TRA BẢN NX (Nguyên tử) ===")
r.flushdb()
wins_nx = race(lambda i: hold_nx("c1", "A1", f"user{i}"))
print("winners (luôn luôn là 1):", sum(wins_nx))
HOLD = r.register_script("""
Problems Output Debug Console Terminal Ports
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
PS D:\year 3\CSDL\New folder>
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
=== KIỂM TRA BẢN NAIVE (GET rồi SET) ===
winners (mong đợi 1, nhưng thực tế sẽ thấy nhiều hơn): 11
=== KIỂM TRA BẢN NX (Nguyên tử) ===
winners (luôn luôn là 1): 1
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
powershell
Python
docker
2 files and 0 cells to analyze
Ln 195, Col 64 Spaces: 4 UTF-8 CRLF Python Sign In Python 3.14.6 Go Live

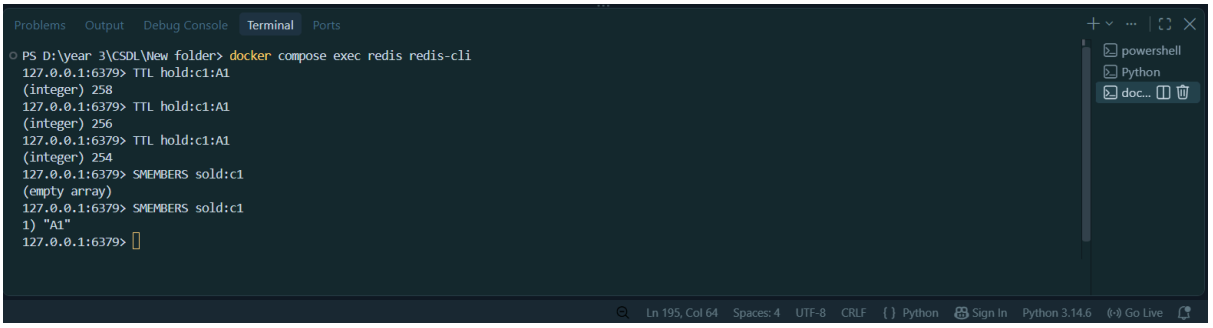

File Edit Selection View Go
New folder
Explorer
docker-compose.yaml
ticketrush.py
main.py
New folder
.vscode
chap3.pdf
chap4.pdf
chap5.pdf
chap6.pdf
chap7.pdf
chap8.pdf
docker-compose.yaml
main.py
mcq.pdf
ticketrush.py
Outline
Timeline
ticketrush.py > ...
def hold(event, seat, user, ttl=300):
return bool(HOLD(keys=[f"hold:{event}:{seat}", f"sold:{event}"], args=[user, seat, ttl]))
def confirm(event, seat, user):
return bool(CONFIRM(keys=[f"hold:{event}:{seat}", f"sold:{event}"], args=[user, seat]))
def release(event, seat, user):
return bool(RELEASE(keys=[f"hold:{event}:{seat}"], args=[user]))
def seat_map(event, seats):
sold = r.smembers(f"sold:{event}")
return {
s: "sold" if s in sold else "held" if r.exists(f"hold:{event}:{s}") else "free"
for s in seats
}
if __name__ == "__main__":
print("\n=== KIỂM TRA KỊCH BẢN BƯỚC 2 ===")
r.flushdb()
print("linh giữ A1:", hold("c1", "A1", "linh")) # True
print("nam giữ A1:", hold("c1", "A1", "nam")) # False, linh đang giữ
print("nam xác nhận A1:", confirm("c1", "A1", "nam")) # False, nam không sở hữu quyền giữ
print("linh xác nhận A1:", confirm("c1", "A1", "linh")) # True, đã bán thành công
print("nam giữ lại A1:", hold("c1", "A1", "nam")) # False, ghế đã bán
print("Trạng thái sơ đồ ghế:", seat_map("c1", ["A1", "A2"])) # {'A1': 'sold', 'A2': 'free'}
Problems Output Debug Console Terminal Ports
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
=== KIỂM TRA KỊCH BẢN BƯỚC 2 ===
linh giữ A1: True
nam giữ A1: False
nam xác nhận A1: False
linh xác nhận A1: True
nam giữ lại A1: False
Trạng thái sơ đồ ghế: {'A1': 'sold', 'A2': 'free'}
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
powershell
Python
docker
Ln 195, Col 64 Spaces: 4 UTF-8 CRLF Python Sign In Python 3.14.6 Go Live


Explorer
New folder
.vscode
chap3.pdf
chap4.pdf
chap5.pdf
chap6.pdf
chap7.pdf
chap8.pdf
docker-compose.yaml
main.py
mcq.pdf
ticketrush.py
Outline
Timeline
docker-compose.yaml
ticketrush.py
main.py
ticketrush.py > ...
print("linh giữ A1:", hold("c1", "A1", "linh")) # True
print("nam giữ A1:", hold("c1", "A1", "nam")) # False, linh đang giữ
print("nam xác nhận A1:", confirm("c1", "A1", "nam")) # False, nam không sở hữu quyền giữ
print("linh xác nhận A1:", confirm("c1", "A1", "linh")) # True, đã bán thành công
print("nam giữ lại A1:", hold("c1", "A1", "nam")) # False, ghế đã bán
print("Trạng thái sơ đồ ghế:", seat_map("c1", ["A1", "A2"])) # {'A1': 'sold', 'A2': 'free'}
bước 3
def buy_general(event, user):
left = r.decr(f"stock:{event}")
if left < 0:
r.incr(f"stock:{event}") # undo: nothing was left
return False
r.sadd(f"buyers:{event}", user)
return True
r.flushdb()
r.set("stock:g1", 100)
ok = race(lambda i: buy_general("g1", f"user{i}"), n=500)
print(sum(ok), r.get("stock:g1"), r.scard("buyers:g1")) # 100 0 100
--- BƯỚC 4: GIỚI HẠN TẦN SUẤT (RATE LIMITING) ---
1. Fixed Window (Cửa sổ cố định)
def allow_fixed(user, limit=5, window=10):
key = f"rl:fixed:{user}:{int(time.time() // window)}"
pipe = r.pipeline()
pipe.incr(key)
Problems
Output
Debug Console
Terminal
Ports
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
linh giữ A1: True
nam giữ A1: False
nam xác nhận A1: False
linh xác nhận A1: True
nam giữ lại A1: False
Trạng thái sơ đồ ghế: {'A1': 'sold', 'A2': 'free'}
100 0 100
powershell
Python
docker
Ln 195, Col 64
Spaces: 4
UTF-8
CRLF
Python
Sign In
Python 3.14.6
Go Live


Explorer
New folder
.vscode
chap3.pdf
chap4.pdf
chap5.pdf
chap6.pdf
chap7.pdf
chap8.pdf
docker-compose.yaml
main.py
mcq.pdf
ticketrush.py
Outline
Timeline
docker-compose.yaml
ticketrush.py
main.py
ticketrush.py > ...
1. Fixed Window (Cửa sổ cố định)
def allow_fixed(user, limit=5, window=10):
key = f"rl:fixed:{user}:{int(time.time() // window)}"
pipe = r.pipeline()
pipe.incr(key)
pipe.expire(key, window, nx=True) # Chỉ đặt TTL ở lần đếm đầu tiên
count, _ = pipe.execute()
return count <= limit
2. Sliding Window (Cửa sổ trượt dùng Lua script)
SLIDING = r.register_script("""
local now, window, limit = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])
redis.call('ZREMRANGEBYSCORE', KEYS[1], 0, now - window)
if redis.call('ZCARD', KEYS[1]) < limit then
redis.call('ZADD', KEYS[1], now, ARGV[4])
redis.call('PEXPIRE', KEYS[1], window)
return 1
end
return 0
""")
def allow_sliding(user, limit=5, window_ms=10_000):
now = int(time.time() * 1000)
return bool(SLIDING(keys=[f"rl:sliding:{user}"], args=[now, window_ms, limit, uuid.uuid4().hex]))
Problems
Output
Debug Console
Terminal
Ports
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
100 0 100
=== KIỂM TRA FIXED WINDOW ===
[True, True, True, True, True, False, False, False]
=== KIỂM TRA ĐIỂM YẾU RANH GIỚI CỦA FIXED WINDOW ===
Fixed window (Cho phép cả 10 request vượt giới hạn qua ranh giới): [True, True, True, True, True, True, True, True, True, True]
=== KIỂM TRA SLIDING WINDOW ===
Sliding window (Đợt thứ hai sẽ bị chặn đứng): [True, True, True, True, True, False, False, False, False, False]
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
=== KIỂM TRA BẢN NAIVE (GET rồi SET) ===
powershell
Python
docker
Ln 195, Col 64
Spaces: 4
UTF-8
CRLF
Python
Sign In
Python 3.14.6
Go Live


Explorer
New folder
.vscode
chap3.pdf
chap4.pdf
chap5.pdf
chap6.pdf
chap7.pdf
chap8.pdf
docker-compose.yaml
main.py
mcq.pdf
ticketrush.py
Outline
Timeline
docker-compose.yaml
ticketrush.py
main.py
ticketrush.py > ...
print('BƯỚC 5')
def join_queue(event, user):
r.zadd(f"queue:{event}", {user: time.time()}, nx=True) # NX: re-joining keeps your place
return r.zrank(f"queue:{event}", user) + 1
def admit(event, n):
return [user for user, _ in r.zpopmin(f"queue:{event}", n)]
for name in ["u1", "u2", "u3"]:
print(name, join_queue("c1", name)) # positions 1, 2, 3
print(join_queue("c1", "u1")) # still 1
print(admit("c1", 2)) # ['u1', 'u2']
print(join_queue("c1", "u3")) # 1
print('===BƯỚC 6===')
import random
r.flushdb()
SEATS = [f"S{i}" for i in range(1, 21)]
def buyer(i):
user = f"user{i}"
Problems
Output
Debug Console
Terminal
Ports
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
BƯỚC 5
u1 1
u2 2
u3 3
1
['u1', 'u2']
1
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
=== KIỂM TRA BẢN NAIVE (GET rồi SET) ===
winners (mong đợi 1, nhưng thực tế sẽ thấy nhiều hơn): 12
=== KIỂM TRA BẢN NX (Nguyên tử) ===
powershell
Python
docker
Ln 195, Col 64
Spaces: 4
UTF-8
CRLF
Python
Sign In
Python 3.14.6
Go Live


Explorer
New folder
.vscode
chap3.pdf
chap4.pdf
chap5.pdf
chap6.pdf
chap7.pdf
chap8.pdf
docker-compose.yaml
main.py
mcq.pdf
ticketrush.py
Outline
Timeline
docker-compose.yaml
ticketrush.py
main.py
ticketrush.py > ...
print('===BƯỚC 6===')
import random
r.flushdb()
SEATS = [f"S{i}" for i in range(1, 21)]
def buyer(i):
user = f"user{i}"
for seat in SEATS:
if hold("c1", seat, user):
time.sleep(random.random() * 0.05) # paying
if confirm("c1", seat, user):
return seat
return None
got = [s for s in race(buyer, n=100) if s]
print(len(got), len(set(got)), r.scard("sold:c1")) # 20 20 20
Problems
Output
Debug Console
Terminal
Ports
PS D:\year 3\CSDL\New folder> & C:\Users\PC\anaconda3\python.exe "d:/year 3/CSDL/New folder/ticketrush.py"
Sliding window (Đợt thứ hai sẽ bị chặn đứng): [True, True, True, True, True, False, False, False, False, False]
BƯỚC 5
u1 1
u2 2
u3 3
1
['u1', 'u2']
1
===BƯỚC 6===
20 20 20
PS D:\year 3\CSDL\New folder>
powershell
Python
docker
Ln 195, Col 64
Spaces: 4
UTF-8
CRLF
Python
Sign In
Python 3.14.6
Go Live


Problems
Output
Debug Console
Terminal
Ports
PS D:\year 3\CSDL\New folder> docker compose exec redis redis-cli
127.0.0.1:6379> TTL hold:c1:A1
(integer) 258
127.0.0.1:6379> TTL hold:c1:A1
(integer) 256
127.0.0.1:6379> TTL hold:c1:A1
(integer) 254
127.0.0.1:6379> SMEMBERS sold:c1
(empty array)
127.0.0.1:6379> SMEMBERS sold:c1
1) "A1"
127.0.0.1:6379>
powershell
Python
doc...
Ln 195, Col 64
Spaces: 4
UTF-8
CRLF
Python
Sign In
Python 3.14.6
Go Live

1. Ở Bước 1, vì sao SET NX EX tránh được race còn GET rồi SET thì không?

- **Bản GET rồi SET (Naive):** Gặp lỗi *check-then-act* (kiểm tra rồi mới ghi). Do hai thao tác này tách biệt nhau, giữa lúc thread A thực hiện GET (thấy ghế trống) và lúc ghi SET, các thread khác cũng thực hiện GET và đều thấy ghế trống. Điều này dẫn đến việc nhiều thread cùng ghi đè thành công (gây ra race condition).
- **Lệnh SET NX EX:** Redis thực hiện việc kiểm tra xem key đã tồn tại chưa (NX), đặt thời hạn (EX), và ghi giá trị **trong cùng một lệnh nguyên tử duy nhất (atomic)**. Tuyệt đối không có thread nào có thể chen ngang vào giữa quá trình này, đảm bảo chỉ có duy nhất một client giành được quyền giữ ghế.

2. Điều gì xảy ra với một ghế nếu khách giữ ghế rồi đóng trình duyệt? Tính năng nào của Redis xử lý việc đó?

- Nếu khách giữ ghế rồi đóng trình duyệt mà không hoàn tất thanh toán, ghế sẽ được giữ tạm thời cho đến khi hết thời gian.
- **Tính năng xử lý: TTL (Time To Live / thời hạn hết hạn với tham số EX)** của Redis. Khi hết thời gian quy định (ví dụ 300 giây), Redis tự động xóa key hold, làm cho ghế tự động trở về trạng thái trống (free) mà không cần bất kỳ cron job hay tiến trình dọn dẹp thủ công nào.

3. Ở Bước 2, điều gì sai nếu confirm không so sánh chủ sở hữu của việc giữ ghế?

- Nếu lệnh confirm không kiểm tra chủ sở hữu (user), bất kỳ ai cũng có thể xác nhận hoặc can thiệp vào ghế đang được giữ.
- Điều này dẫn đến lỗ hổng bảo mật nghiêm trọng: User B có thể xác nhận mua nhầm hoặc cướp ghế đang được giữ bởi user A, hoặc user B có thể xác nhận sau khi hạn giữ của user A đã hết và một người khác đã kịp giữ ghế đó. Việc so sánh chủ sở hữu trong Lua script đảm bảo chỉ đúng người đang giữ ghế mới có quyền xác nhận thanh toán.

4. Khi nào fixed window là đủ tốt, và khi nào cần sliding window?

- **Fixed window:**
  - *Đủ tốt khi:* Hệ thống ở mức đơn giản, tài nguyên hạn chế, chấp nhận được hiện tượng bùng nổ request ở ranh giới cửa sổ thời gian (tối đa gấp đôi giới hạn trong một khoảnh khắc ngắn).
  - *Ưu điểm:* Tiết kiệm bộ nhớ (chỉ cần 1 bộ đếm cho mỗi user) và thực hiện nhanh (1 round-trip).
- **Sliding window:**
  - *Cần thiết khi:* Yêu cầu kiểm soát tần suất nghiêm ngặt tuyệt đối, không cho phép vượt quá giới hạn ngay cả khi bot dồn dập gửi request đúng vào ranh giới thời gian (chống bot hiệu quả, lưu timestamp trong sorted set).

5. Ở đây Redis là tầng nhanh. Dữ liệu nào bạn còn ghi thêm vào một cơ sở dữ liệu bền vững, và khi nào? (Nghĩ xem chuyện gì xảy ra nếu Redis khởi động lại giữa confirm và lúc ghi bản ghi thanh toán.)

- **Dữ liệu cần ghi vào cơ sở dữ liệu bền vững (Persistent DB như SQL/NoSQL):** Các bản ghi giao dịch chính thức, thông tin đơn hàng, lịch sử thanh toán, và danh sách vé đã phát hành thành công cho khách hàng để lưu trữ lâu dài, phục vụ cho báo cáo tài chính, tra cứu đơn hàng và kiểm toán.
- **Thời điểm ghi:** Ngay sau khi xác nhận thanh toán thành công (confirm).
- **Sự cố khi Redis khởi động lại:** Redis là cơ sở dữ liệu lưu trên RAM (*in-memory*). Nếu Redis khởi động lại đột ngột (hoặc gặp sự cố sập nguồn) giữa lúc confirm và lúc ghi vào DB bền vững, dữ liệu trạng thái trên Redis có thể bị mất (nếu không bật tính năng Persistence như RDB/AOF), gây ra sự lệch lạc giữa hệ thống cache và cơ sở dữ liệu chính. Do đó, cơ sở dữ liệu bền vững đóng vai trò là "chân lý cuối cùng" (source of truth) để đảm bảo an toàn dữ liệu giao dịch.